

# Lecture 7: Reinforcement Learning

Jelle Zuidema

ILLC, Universiteit van Amsterdam  
(based on a slideset by Iris Proff)

Machine Learning and Language Models, 2025

# Plan

What is RL?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

# Types of learning algorithms

- Supervised learning

Learning from labeled examples – neural networks

- Unsupervised learning

Findings patterns in the raw data - clustering

- Semi-supervised learning

Learning from reward signals – Reinforcement learning



# Reinforcement learning

**...formalized by AI to a cognitive model**

## Structural model

Classical conditioning

Instrumental conditioning

Goal directed learning

## Cognitive model

→ Rescorla-Wagner model (1972)

→ Temporal difference learning (Samuel 1959)

→ Q-Learning (Watkins & Dayan 1992)

→ Model-based learning

Reinforcement  
Learning

An Introduction  
second edition  
Richard S. Sutton and Andrew G. Barto

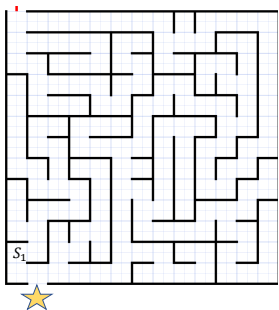


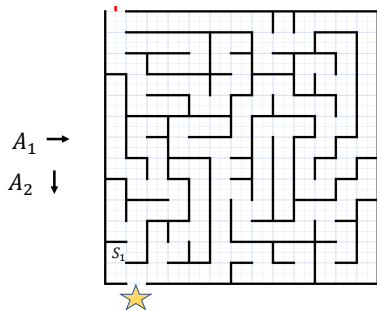
## Determining utility (optimal control theory)



$$U(S) = R(S)$$

 $R_1$  $R_2$

$A_1 \rightarrow$  $A_2 \downarrow$ 

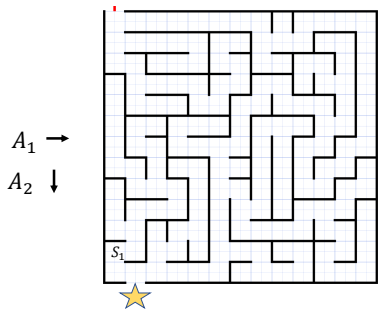


## Question 1

Consider states  $S_2$  and  $S_3$  (which can you reach with action  $A_1$  and  $A_2$ ) and state  $S_4$  (which you can reach from state  $S_3$  by action  $A_1$ ). How valuable are each of these states relative to each other?

Is  $U(S_1) = U(S_3)$ ?

## Determining utility (optimal control theory)



**Bellman equation:**

$$U(S) = R(S) + \gamma * \max_A [U(S')]$$

U: utility

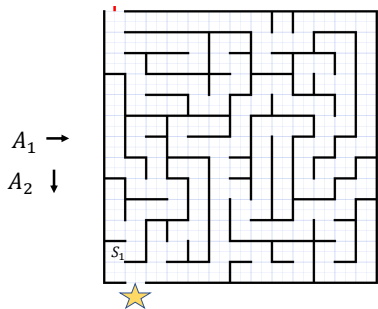
S: state

R: reward

A: action

$\gamma$ : temporal discount factor

## Determining utility (optimal control theory)



**Bellman equation:**

$$U(S) = R(S) + \gamma * \max_A [U(S')]$$

U: utility

S: state

R: reward

A: action

$\gamma$ : temporal discount factor

### Question 2

How do you avoid having to first learn everything about the environment/problem before you can start acting in the world?

# Plan

What is RL?

**Q-learning**

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

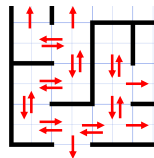
# Q-Learning / temporal-difference learning

How to learn the utility of state-action pairs by trial-and-error?

Learning the Q-value = approximation of utility

$$Q(S, A) = \hat{U}(S, A)$$

Online learning (policy is applied and learned at the same time)



# Q-Learning – update rule

Prediction error

$$\delta = R + \gamma * \max_{A'} (Q_t(S', A')) - Q_t(S, A)$$

Q-value update

$$Q_{t+1}(S, A) = Q_t(S, A) + \alpha * \delta$$

Q-value approximates utility

$$\lim_{t \rightarrow \infty} Q_t(S, A) = U(S, A)$$

Reminder: Bellman equation

$$U(S, A) = R(S, A) + \gamma * \max_{A'} [U(S', A')]$$

S: state

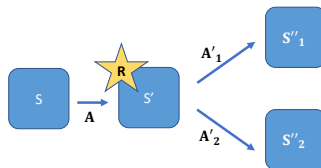
R: reward

A: action

$\gamma$ : temporal discount factor

$\alpha$ : learning rate

U: utility



## Q-learning – action selection policy

Say we are in state  $S$ , with possible actions  $\mathbf{A}_1$  and  $\mathbf{A}_2$ , with  
 $\mathbf{Q}(S, \mathbf{A}_1) = 0.5$  and  $\mathbf{Q}(S, \mathbf{A}_2) = 0.7$

**Which action should we choose?**

## Q-learning – action selection policy

Say we are in state  $S$ , with possible actions  $A_1$  and  $A_2$ , with  
 $Q(S, A_1) = 0.5$  and  $Q(S, A_2) = 0.7$

**Which action should we choose?**

### Question 3

How do you make sure you maximize reward, both in the short term (taking advantage of your knowledge) and in the long term (extending your knowledge)?

## Exploration – exploitation trade-off



exploit what you know

should you...

**or**

explore new options?



“A reinforcement learning agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task.”

Sutton & Barto 2018

## Exploration – exploitation trade-off



exploit what you know

should you...

**or**

explore new options?



“A reinforcement learning agent has to exploit what it has already experienced in order to obtain reward, but it also has to explore in order to make better action selections in the future.

The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task.”

Sutton & Barto 2018

Classic example: [multi-armed bandit](#)

## Multi-armed bandit



*The name comes from imagining a gambler at a row of slot machines ("one-armed bandits"), who has to decide which machines to play, how many times to play each machine and in which order to play them, and whether to continue with the current machine or try a different machine.*

## Q-learning – action selection policy

Say we are in state  $S$ , with possible actions  $A_1$  and  $A_2$ , with

$$Q(S, A_1) = 0.5 \quad \text{and} \quad Q(S, A_2) = 0.7$$

**Which action should we choose, to trade off exploration and exploitation?**

Epsilon greedy: 
$$P(a_i|S) = \begin{cases} \frac{1-\epsilon}{|max_{q_l}(A)|} & \text{for } q_i = max(A) \\ \frac{\epsilon}{|A| - |max_{q_l}(A)|} & \text{else} \end{cases}$$
 best action chosen with  $P = 1 - \epsilon$

Softmax: 
$$P(a_i|S) = \frac{e^{\frac{q_i}{\tau}}}{\sum_A e^{\frac{q_l}{\tau}}}$$
 for  $\tau \rightarrow 0$  greedy, for  $\tau \rightarrow \infty$  random

# Q-Learning – pseudo code

Set  $\alpha \in (0,1]$ , small  $\varepsilon > 0$ ,  $\gamma > 0$

Initialize  $Q(s, a)$  for all  $s \in S, a \in A$  to 0

Initialize  $s$

For number of episodes

While  $s$  is not a terminal state

choose  $a$  from  $s$  using action selection policy

observe  $R$  and  $s'$

update  $Q$

update  $s$  to  $s'$

# Plan

What is RL?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL



# Plan

What is RL?

Q-learning

RL in the brain

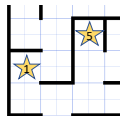
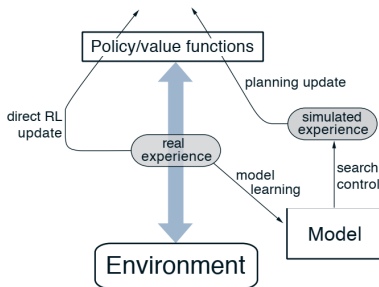
Model-based RL

Extensions of Q-learning

Deep RL



# Model-based reinforcement learning



Model:  $(s, a) \rightarrow (s', r)$

»What does my environment look like? «

Sutton & Barto 2018





# Plan

What is RL?

Q-learning

RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

# Extensions of RL algorithms

- Eligibility traces

update not only last state-action pairs, but all previously visited state-action pairs with decay factor  $\lambda$

- Decaying learning rate

$\alpha$  decreases over time

- Hybrid models

Model-based and model-free component

...

# Plan

What is RL?

Q-learning

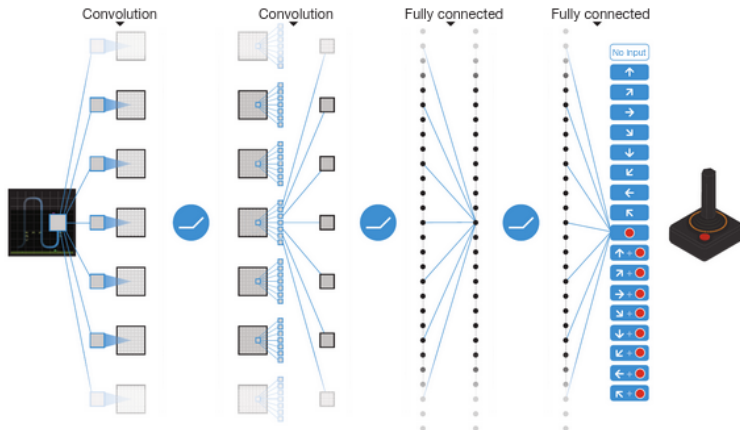
RL in the brain

Model-based RL

Extensions of Q-learning

Deep RL

# Deep Reinforcement Learning



## Deep Q-Learning (Mnih et al. 2015)

- Replace Q-table with a neural network  $Q$ , with weights  $\theta$ ; use it to compute Q-value of current state
- Make a copy of the Q-network (the *target-network*  $\hat{Q}$ ); use it to compute Q-value of target states
- Learn in batches: act in the world (choose actions using  $\epsilon$ -greedy) and gather data
- Optimize Q-network (minimize prediction error) via gradient descent on a batch of observations
- Occasionally copy  $Q$  into  $\hat{Q}$

# Deep Q-Learning (Mnih et al. 2015)

## Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory  $D$  to capacity  $N$

Initialize action-value function  $Q$  with random weights  $\theta$

Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$

**For** episode = 1,  $M$  **do**

Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$

**For**  $t = 1, T$  **do**

With probability  $\epsilon$  select a random action  $a_t$

otherwise select  $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$

Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$

Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$

Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$

Set  $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  with respect to the network parameters  $\theta$

Every  $C$  steps reset  $\hat{Q} = Q$

**End For**

**End For**

# Deep Q-Learning (Mnih et al. 2015)

## LETTER

### Human-level control through deep reinforcement learning

Vedkoymr Mnih<sup>1</sup>, Koray Kavukcuoglu<sup>1\*</sup>, David Silver<sup>2</sup>, Andrei A. Rusu<sup>1</sup>, Joel Veness<sup>1</sup>, Miro G. Bellemare<sup>1</sup>, Alex Graves<sup>1</sup>, Martin Riedmiller<sup>1</sup>, Andrew K. Fiedorowicz<sup>1</sup>, Georg Ostrovski<sup>1</sup>, Stig Petersen<sup>1</sup>, Charles Beattie<sup>1</sup>, Andre Sadiq<sup>1</sup>, Ioannis Antonoglou<sup>1</sup>, Haden Kain<sup>1</sup>, Uthman Kumar<sup>1</sup>, Zoran Wierstra<sup>1</sup>, Shane Legg<sup>1</sup> & Demis Hassabis<sup>1</sup>

The theory of reinforcement learning provides a normative account<sup>1</sup>, deeply rooted in psychological<sup>2</sup> and neuroscience<sup>3</sup> perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use those to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems<sup>4</sup>, the former evidenced by a wealth of neural data revealing intricate parallels between the plastic signals emitted by dopamine neurons and temporal difference reinforcement learning algorithms<sup>5</sup>. While reinforcement learning agents have achieved some successes in a variety of domains<sup>6,7</sup>, their applicability has previously been limited to domains in which useful features can be handcrafted, or the domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks<sup>8,9</sup> to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games<sup>10</sup>. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games player across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learning to excel at a diverse array of challenging tasks.

We set out to create a single algorithm that would be able to develop a wide range of competencies on a varied range of challenging tasks—a central goal of general artificial intelligence<sup>11</sup> that has eluded previous attempts<sup>12</sup>. To achieve this, we developed a novel agent, a deep Q-network (DQN), which is able to combine reinforcement learning with a class of artificial neural networks<sup>13</sup> to learn a value function. Notably, recent advances in deep neural networks<sup>14,15</sup>, in which several layers of nodes are used to build up progressively more abstract representations of the data, have made it possible for artificial neural networks to learn concepts such as object categories directly from raw sensory data. We use particularly successful methods and architectures, the deep convolutional network<sup>16</sup>, which uses hierarchical layers of feed-forward filters to extract the features of receptive fields—inspired by Hubel and Wiesel’s seminal work on bottom-up processing of early visual cortex<sup>17</sup>—thereby exploiting the local spatial correlations present in images, and building in invariances to natural transformations such as changes of viewpoint or scale.

We consider tasks in which the agent interacts with an environment through a sequence of observations, actions and rewards. The goal of the

agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s, a) = \max_{a'} \mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid s^0 = s, a^0 = a, s_1 = s', \dots \right]$$

which is the maximum mean of rewards,  $r$ , discounted by  $\gamma$  at each time step  $t$ , achievable by a behaviour policy  $\pi = \rho(a|s)$  after making an observation ( $s$ ) and taking an action ( $a$ ) (see Methods<sup>18</sup>).

Reinforcement learning is known to be suitable or even to diverge when a nonlinear function, approximator such as a neural network, is used to represent the action-value (also known as  $Q$ ) function<sup>19</sup>. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to  $Q$  may quickly change the policy and therefore change the data distribution, and the correlations between the action values ( $Q$ ) and the target values  $r + \gamma \max_{a'} Q(s', a')$ . We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay<sup>20,21</sup> that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values ( $Q$ ) towards target values that are only partially updated, thereby reducing correlations with the target.

While other stable methods exist for training neural networks in the reinforcement learning setting, such as neural fitted Q iteration<sup>22</sup>, these methods involve the optimization of networks over sums on thousands of iterations. Conversely, these methods, unlike our algorithm, are too inefficient to be used successfully with large neural networks. We parameterize an approximate value function ( $Q(s, a)$ ) using the deep convolutional neural network shown in Fig. 1, in which  $\theta$  denotes the parameters (that is, weights) of the  $Q$ -network at iteration  $t$ . To perform experience replay we store the agent’s experience in a buffer. At each time step  $t$  it is a data set  $D_t = \{s_{t-1}, a_{t-1}\}$ . During learning, we apply Q-learning updates, or use the (reminiscent of) experience replay<sup>20,21</sup>.  $Q(s_{t-1}, a_{t-1})$  is chosen uniformly at random from the pool of stored samples. The Q-learning update at iteration  $t$  uses the following loss function

$$L_t(\theta) = \mathbb{E}_{(s, a) \sim D_t} \left[ \frac{1}{2} \left( r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)^2 \right]$$

in which  $\theta$  is the discount factor determining the agent’s horizon,  $\theta$  are the parameters of the  $Q$ -network at iteration  $t$  and  $\theta'$  are the network parameters used to compute the target at iteration  $t$ . The target network parameters  $\theta'$  are only updated with the  $Q$ -network parameters ( $\theta$ ) every  $C$  steps and are held fixed between individual updates (see Methods).

To evaluate our DQN agent, we took advantage of the Atari 2600 platform, which offers a diverse array of tasks ( $n = 49$ ) designed to be

## NEWS & VIEWS

### ARTIFICIAL INTELLIGENCE

### Learning to see and act

An artificial–intelligence system uses machine learning from massive training sets to teach itself to play 49 classic computer games, demonstrating that it can adapt to a variety of tasks. SEE LETTER P. 529

#### BERNARD SCHÖDL

Improvements in our ability to process large amounts of data have led to progress in many areas of science, not least artificial intelligence (AI). With advances in machine learning has come the development of machines that can learn intelligent behaviour directly from data, rather than being explicitly programmed to exhibit such behaviour. For instance, the advent of ‘big data’ has resulted in systems that can recognize objects or sounds with considerable precision. On page 529 of this issue, Mnih et al. describe an agent that uses large data sets to teach itself how to play 49 classic Atari 2600 computer games by looking at the pixels and learning actions that increase the game score. These are professional games played in many instances – a remarkable example of the progress being made in AI.

In machine learning, systems are trained to infer patterns from observational data. The human that successfully interprets simple type-1 patterns, a mapping between input and output, can be learnt through a process called supervised learning. A supervised learning system is given training data consisting of example inputs and the corresponding outputs, and comes up with a

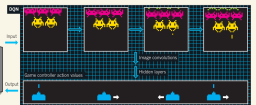
model to explain those data (a process called function approximation). It does this by choosing from a class of model specified by the system’s designer. Designing this class is an art in its own right and complexity should reflect the amount of training data available, and its content should reflect ‘prior knowledge’ that the designer of the system considers useful for the problem at hand. If all this is done well, the inferred model will then apply not only to the training set, but also to other data that adhere to the same underlying pattern.

The rapid growth of data sets means that machine learning can now use complex model classes and large high-resolution training data sets. Such problems are usually characterized by several factors: the data are multidimensional; the underlying patterns are complex (for instance, it might be nonlinear or changeable); and the designer has only weak prior knowledge about the problem – in particular, a mechanistic understanding of learning.

The human that successfully interprets simple type-1 patterns, a mapping between input and output, can be learnt through a process called supervised learning. A supervised learning system is given training data consisting of example inputs and the corresponding outputs, and comes up with a

we often learn without a ‘supervisor’ telling us the outputs of a hypothetical input–output function. Here, ‘reinforcement’ has a central role in learning behaviours from weaker supervision. Machine learning adopted this idea to develop reinforcement learning algorithms, in which supervisor takes the form of a numerical ‘reward signal’, and the goal is for the system to learn a policy that, given the current state, determines which action to pick to maximize an accumulated future reward.

Mnih et al. use a form of reinforcement learning known as Q-learning<sup>23</sup> to teach systems to play a set of 49 vintage video games, learning how to increase the game score as a numerical reward. In Q-learning,  $Q(s, a)$  represents the accumulated future reward,  $Q$ , if in state  $s$  the system performs action  $a$ , and subsequently follows an optimal policy. The system tries to approximate  $Q^*$  by using an artificial neural network – a function approximator loosely inspired by biological neural networks – called a deep Q-network (DQN). The DQNs split the pixels from four consecutive game screens to processed by a network of hidden layers of computations, which extract more and more specialized visual features to help approximate the complex



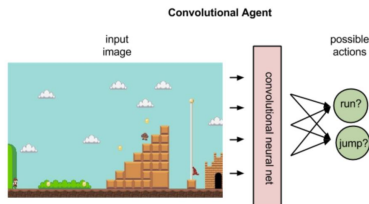
**Figure 1 | Computer games.** Mnih et al. have designed an artificial–intelligence system, using a large Q-network (DQN) that learns how to play 49 video games. The DQNs analyse a sequence of four game screens (input) and approximate, using a neural network, the action values (Q) that the system has to optimize. The action values are then used to select the best possible course of subsequent actions. The action values of the DQN are used to select the best possible course of subsequent actions. The action values of the DQN are used to select the best possible course of subsequent actions.

\*Senior Design Lead, Google DeepMind, London SE26 6UA, UK.

These authors contributed equally to this work.

# Deep Reinforcement Learning

Learn Q-values with deep neural nets



[A nice tutorial on deep reinforcement learning](https://wiki.pathmind.com/deep-reinforcement-learning)

<https://wiki.pathmind.com/deep-reinforcement-learning>